

PROYECTO 8

Del sistema experto al context graph

Sistema experto moderno sobre la saga Final Fantasy usando 3DJuegos, context graph y OpenRouter.

RESUMEN

El proyecto construye un context graph con procedencia, fecha y confianza para responder preguntas sobre Final Fantasy. Combina scraping, extracción de tripletas por reglas y por LLM, construcción del grafo y consulta final con OpenRouter sobre evidencia recuperada.

CONTENIDO

- Objetivos del sistema experto y justificación del dominio
- Arquitectura del pipeline y scripts utilizados
- Construcción del context graph y comparación con baseline
- Preguntas, respuestas citadas y ejemplo multi-hop
- Conclusión sobre context graph frente a RAG vectorial

Autor: SuliAragon

Módulo: Modelos de Inteligencia Artificial

Final Fantasy como caso de estudio

Este proyecto toma la saga Final Fantasy como dominio porque es un entorno claramente relacional: hay entregas principales, remakes, plataformas, fechas de lanzamiento, géneros y desarrolladores. Esa estructura permite formular preguntas que no dependen solo de recuperar texto, sino de conectar hechos entre sí.

La fuente principal ha sido 3DJuegos, usando tanto el universo general de Final Fantasy como fichas concretas de juegos clave como Final Fantasy VII Remake, Final Fantasy VII Rebirth y Final Fantasy XVI.

OBJETIVO DEL SISTEMA

- Identificar juegos, plataformas, géneros, desarrolladores y fechas.
- Representar relaciones como `is_remake_of`, `developed_by`, `published_by` y `available_on`.
- Responder preguntas simples y multi-hop con citas de fuente y fecha.
- Combinar un grafo con un LLM como motor de inferencia final.

ARQUITECTURA

- `scraping.py` descarga páginas en Markdown a través de `r.jina.ai`.
- `obtain_triplets.py` extrae tripletas mediante reglas sobre el texto.
- `obtain_triplets_llm.py` extrae tripletas con OpenRouter en fichas clave.
- `merge_triplets.py` combina reglas y LLM.
- `context_graph.py` construye el grafo y su visualización parcial.
- `ask_llm.py` recupera evidencia del grafo y usa OpenRouter para responder.

Modelado, metadatos y resultados

El sistema genera dos versiones del conocimiento: un baseline sin metadatos y un context graph enriquecido con procedencia, fecha de extracción, validez temporal y confianza. Eso permite justificar cada respuesta y filtrar hechos según contexto temporal.

- Aristas baseline: 626
- Aristas del context graph híbrido: 653
- Nodos del grafo final: 450
- Tripletas añadidas mediante LLM: 39
- Aristas con fecha de vigencia: 152
- Aristas con fuente trazable: 653

Las tripletas generadas por LLM han sido especialmente útiles para relaciones semánticas como `is_remake_of`, `developed_by`, `published_by` y `has_type`, mientras que la extracción por reglas ha servido como baseline reproducible y como red de seguridad cuando la API no aporta cobertura adicional.

SISTEMA EXPERTO CLÁSICO FRENTE A CONTEXT GRAPH

- Base de conocimiento: el grafo de tripletas.
- Motor de inferencia: recuperación de evidencia más respuesta del LLM.
- Interfaz: scripts de consulta `ask.py` y `ask_llm.py`.
- Capa de contexto: `source`, `fecha_extraccion`, `valido_desde`, `valido_hasta` y `confianza`.

Context graph parcial de Final Fantasy

La figura muestra una parte representativa del grafo. Se han limitado los nodos para mantener la legibilidad y enseñar relaciones visibles entre juegos, géneros y universos del dominio.

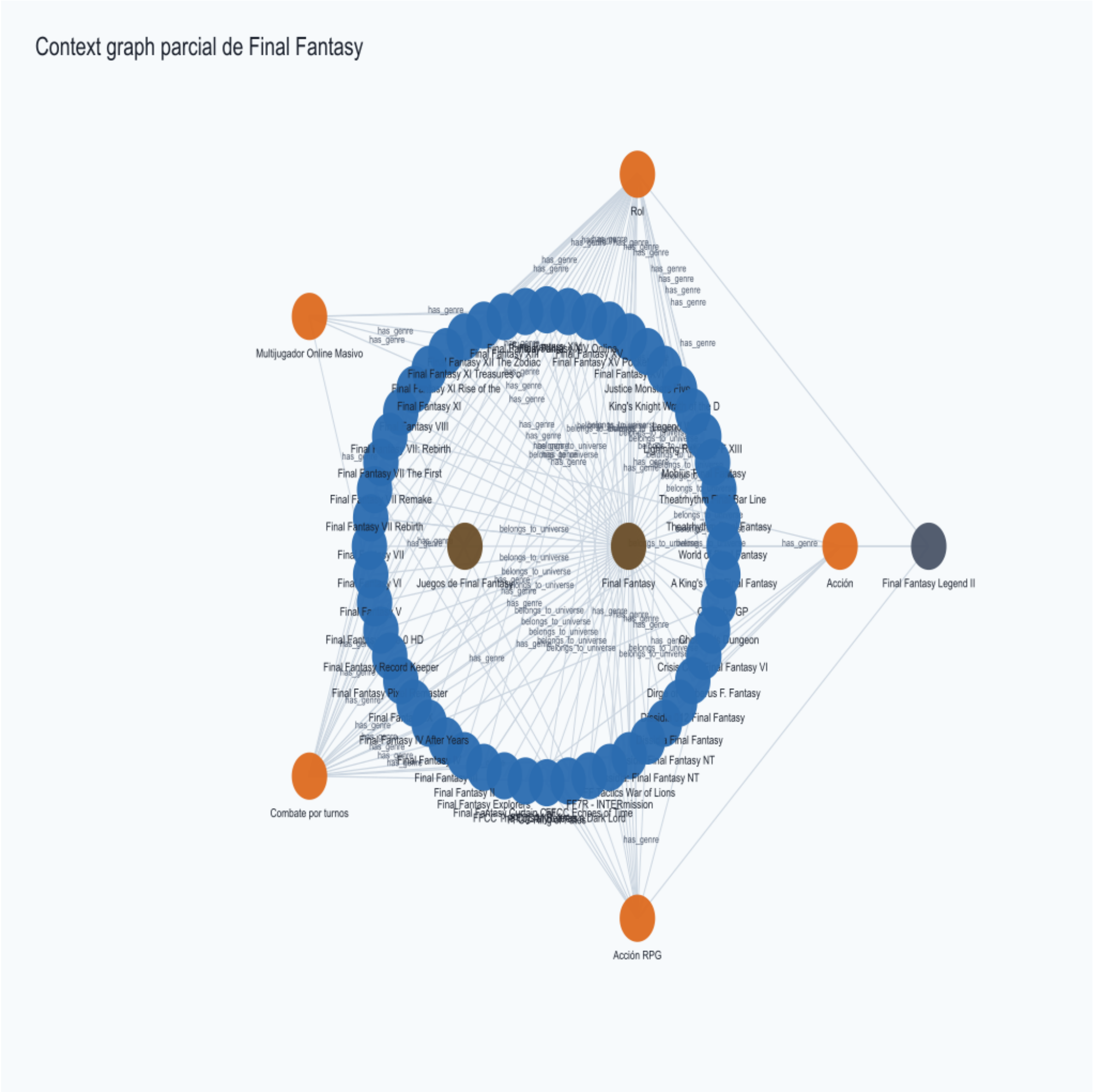


Figura 1. Visualización parcial del context graph construido a partir de Final Fantasy y 3DJuegos.

Consulta temporal sobre PS5

Pregunta: ¿Qué juegos de Final Fantasy salieron en PS5 después de 2020?

- Final Fantasy XVI, con salida en PS5 el 22 de junio de 2023.
Fuente: <https://www.3djuegos.com/universo/0f0f0f0/11/final-fantasy/>
- Final Fantasy VII: Rebirth, con fecha de lanzamiento el 29 de febrero de 2024 y evidencia de disponibilidad en PS5. Fuente: <https://www.3djuegos.com/juegos/final-fantasy-vii-rebirth/>

Esta pregunta combina la relación `available_on` con `released_on` y aplica un filtro temporal posterior a 2020. Es un buen ejemplo de por qué el contexto estructurado mejora frente a recuperar solo texto parecido.

Consulta sobre remakes

Pregunta: ¿Qué entregas de Final Fantasy aparecen como remakes?

- Final Fantasy VII Remake aparece como remake de Final Fantasy VII.
Fuente: <http://www.3djuegos.com/juegos/final-fantasy-vii-remake/>

Remake + desarrollador + juego original

Pregunta: ¿Qué remake de Final Fantasy desarrollado por Square Enix aparece en nuestras fuentes y de qué juego original proviene?

Ruta de razonamiento en el grafo:

- Final Fantasy VII Remake -> developed_by -> Square Enix
- Final Fantasy VII Remake -> is_remake_of -> Final Fantasy VII

Respuesta del sistema:

- El remake identificado es Final Fantasy VII Remake.
- Proviene del juego original Final Fantasy VII.
- Está desarrollado por Square Enix.
- Fuente principal:
<http://www.3djuegos.com/juegos/final-fantasy-vii-remake/>

Esta es la pregunta que mejor representa el objetivo del proyecto, porque no se resuelve con un único hecho sino conectando varias relaciones del context graph y dejando que el LLM redacte la respuesta final usando solo esa evidencia.

COMPARACIÓN

Baseline frente a context graph

- El baseline conserva relaciones sujeto-predicado-objeto, pero no justifica el origen exacto de cada hecho.
- El context graph añade procedencia, fecha y confianza por arista.
- Gracias a esos metadatos se pueden responder preguntas temporales con citas verificables.
- La versión final permite además recuperar evidencia y usar un LLM como motor de inferencia sin dejarle inventar contexto.

ESTRATEGIA DE CONTROL

- Usar Markdown de r.jina.ai para simplificar el scraping.
- Mantener una extracción por reglas como baseline reproducible.
- Complementar con OpenRouter solo en fichas clave.
- Obligar a que ask_llm.py responda únicamente con evidencia recuperada del grafo.
- Citar siempre fuente y fecha en la respuesta final.

Context graph o RAG vectorial

Un RAG vectorial bastaría cuando solo necesitamos recuperar fragmentos relevantes de texto y las relaciones entre entidades no son el núcleo del problema. En cambio, un context graph merece la pena cuando importan las conexiones entre hechos, la procedencia, la vigencia temporal y las preguntas multi-hop.

- RAG vectorial: útil para recuperar texto sin demasiada estructura relacional.
- Context graph: útil cuando la relación entre entidades cambia la respuesta.
- LLM acotado por evidencia: útil para redactar respuestas claras sin renunciar a la trazabilidad.

En este proyecto, Final Fantasy ha servido como dominio manejable y muy relacional para demostrar la idea central del enunciado: el conocimiento experto no se deja dentro del modelo, sino alrededor del modelo, en forma de contexto verificable.

Conclusión

La versión final combina scraping, extracción estructurada, metadatos de contexto, visualización del grafo y un LLM que responde usando evidencia recuperada. El resultado es un sistema más explicable, más trazable y más fiel al patrón moderno de sistema experto que pide la práctica.